

AdoptAR

Uso de Inteligencia Artificial en el Desarrollo del Backend

Cómo usé la IA como mentor: decisiones, fricciones y aprendizajes reales

Emiliano Cozzolino

Tecnicatura Superior en Desarrollo de Software — Backend 2026

Proyecto AdoptAR · v1.0.0 → v1.9.0

Resumen General

El desarrollo del backend de AdoptAR utilizó IA como mentor técnico, no como generador de código. El flujo de trabajo consistió en razonar cada decisión antes de implementarla, con la IA formulando preguntas antes de dar respuestas. El código fue siempre la consecuencia del razonamiento, nunca el punto de partida.

Este documento reúne el proceso completo en dos etapas. La primera (v1.0.0 → v1.5.0) cubre la construcción inicial de la API: arquitectura, autenticación, testing, despliegue y refactor. La segunda (v1.5.0 → v1.9.0) cubre las features incrementales sobre esa base: manejo global de errores, paginación, filtros, estadísticas, promoción de usuarios y recuperación de contraseña.

Parte I — Construcción de la Base (v1.0.0 → v1.5.0)

1. Cómo se configuró la interacción con la IA

Desde el principio se definió un conjunto de instrucciones para que la IA actuara como mentor educativo y no como generador de código. Esto fue determinante para que el aprendizaje fuera real y no una simple delegación de tareas.

Diez directrices funcionaron como base: Mentor Educativo, Generador de Desafíos, Arquitecto de Rutas, Guía en Nuevas Tecnologías, Simulador de Pair Programming, Senior de Confianza, Socio de Brainstorming, Especialista en Testing, Detective de Debugging y Entrevistador Técnico.

La filosofía central era obligar a pensar primero. Si algo era fácil, la dificultad aumentaba. Si se pedía la respuesta directa, primero había que intentarlo de forma independiente.

2. Áreas de mayor interacción

2.1 Planificación y arquitectura

Esta fue la fase con más preguntas y donde la IA aportó más valor como mentor. Antes de escribir código fue necesario tomar decisiones de diseño que impactarían todo el proyecto: definición de entidades y relaciones (Usuario, Animal, Solicitud), elección de tipos de dato por campo, método HTTP por endpoint (PATCH vs PUT), separación de responsabilidades entre routers, services y schemas, y la evaluación de si implementar una capa DAL.



Decisión más importante de esta fase

Usar una sola tabla Usuario con campo rol ENUM en lugar de dos tablas separadas. La pregunta que guió la conclusión fue: ¿en qué se diferencian estructuralmente un admin y un adoptante? La respuesta se razonó de forma independiente.

2.2 Autenticación y JWT

El concepto de JWT fue el primero estudiado en profundidad, entendiéndolo antes de implementarlo. Las preguntas centrales fueron qué contiene un token y cómo se verifica, por

qué incluir el rol en el payload en lugar de consultarlo en la base de datos, la diferencia entre autenticación y autorización, y por qué un login fallido devuelve siempre el mismo mensaje de error.

Aprendizaje clave

El mensaje de error genérico ("Credenciales inválidas") para ambos casos no es descuido: es seguridad. Si el mensaje diferenciara los dos casos, un atacante podría saber si un email existe en el sistema.

2.3 Testing — el área más compleja

El testing fue la etapa con más fricción del proyecto, con tres problemas concretos resueltos en secuencia.

Fricción 1 — Tests que funcionaban solos pero fallaban juntos: cada archivo de tests tenía su propia base de datos SQLite y su propio override de dependencias; al correr en paralelo, los overrides se pisaban entre sí. La pregunta guía fue ¿qué diferencia hay entre correr un test solo y correrlos todos?, lo que llevó a identificar el estado compartido como causa raíz y a centralizar la configuración en conftest.py.

Fricción 2 — DB de tests con datos del run anterior: los archivos test.db y test_animales.db no estaban en el .gitignore y se subieron al repositorio, generando fallos por duplicados al encontrar datos de runs previos. Se resolvió borrando los archivos y agregando las entradas correspondientes al .gitignore.

Fricción 3 — Orden de ejecución de fixtures: el fixture setup_db que creaba las tablas se ejecutaba en un momento distinto al de los helpers que insertaban datos, por lo que las tablas no existían aún. Se resolvió asegurando autouse=True y el scope correcto en el fixture.

Lección del testing

Los tests de integración que usan base de datos requieren un nivel de aislamiento que no es trivial. Configurar bien el entorno de testing tomó más tiempo que escribir los tests en sí.

2.4 Git y GitHub — conflictos y errores de flujo

El Git Flow fue otro punto de fricción recurrente, especialmente al inicio del proyecto. El error más repetido fue committear directamente en develop sin estar en la feature branch correspondiente, lo cual dejaba el PR de la feature vacío al ser develop y la rama idénticas. En otras ocasiones, el PR se abrió apuntando a main en lugar de develop, detectado visualmente en GitHub antes de mergear.

Hábito incorporado

Verificar siempre la rama activa con git branch antes de committear. La rama activa aparece marcada con asterisco. Parece obvio, pero en la práctica es el error más común en Git Flow cuando se trabaja en solitario.

2.5 Swagger y OAuth2PasswordBearer

El botón Authorize de Swagger fue un punto de confusión prolongado. OAuth2PasswordBearer generaba un formulario con campos username y password, mientras que el endpoint de login recibía JSON con email y password; la incompatibilidad hacía fallar el Authorize de forma sistemática. La solución fue reemplazar OAuth2PasswordBearer por HTTPBearer, un cambio de tres líneas en deps.py que requirió primero entender la diferencia entre ambos esquemas de seguridad. Mientras Swagger no funcionaba correctamente, Postman cubrió las pruebas que requerían autenticación.

2.6 Cloudinary e imágenes

La integración con Cloudinary fue conceptualmente simple pero con un obstáculo técnico: Thunder Client no permite enviar archivos en su versión gratuita, lo que obligó a instalar Postman específicamente para probar la subida de imágenes. El aprendizaje arquitectónico más relevante de esta etapa fue entender por qué la API debe actuar como intermediaria con Cloudinary y nunca exponer las credenciales al frontend.

3. Mapa de fricción por área

La siguiente tabla resume, área por área, el nivel de fricción de ambas etapas del proyecto, el principal obstáculo encontrado y cómo se resolvió.

Área	Fricción	Principal obstáculo	Resolución
Arquitectura y diseño	Baja	Definir entidades y relaciones	Preguntas del mentor guiaron el razonamiento
JWT y autenticación	Baja	Entender el payload y la verificación	Estudio conceptual antes de implementar
CRUD de animales	Baja	Soft delete y actualizaciones parciales	PATCH + exclude_unset resolvió limpio
Solicitudes (lógica)	Media	Reglas de negocio complejas	Separar validaciones en orden correcto
Testing	Alta	DB de tests compartida entre archivos	conftest.py centralizado
Git Flow	Media	Commitear en rama incorrecta	Hábito: git branch antes de commitear
Swagger / HTTPBearer	Media	OAuth2 incompatible con JSON login	Reemplazar por HTTPBearer (3 líneas)
Cloudinary	Baja	Thunder Client no soporta archivos	Postman para pruebas de subida
Deploy en Render	Baja	DATABASE_URL local no funciona	PostgreSQL en Render + External URL
Alembic (fase 1)	Baja	Entender create_all vs migraciones	Eliminar create_all, usar alembic upgrade
Alembic (fase 2)	Media	Múltiples heads divergentes	alembic merge heads + verificar antes de migrar

Merge develop → main	Media	Conflictos en archivos de Alembic	Resolución manual en VS Code antes del PR
Recuperación de password	Media	logger.info no visible en uvicorn	Debug con prints + email_service como capa

4. Patrones de interacción con la IA

A lo largo del desarrollo se identificaron cuatro patrones distintos de interacción con el mentor según la situación.

Patrón	Cómo iniciaba el estudiante	Cómo respondía el mentor
Exploración	¿Qué es X y cómo funciona?	Explicación con analogías + pregunta de verificación
Diseño	¿Cómo debería hacer X?	Devolvía la pregunta: "¿Qué pensás vos?"
Validación	Hice X así, ¿está bien?	Validaba con razonamiento o señalaba el error sin resolverlo
Debugging	X no funciona, este es el error	Hacía preguntas hasta que el estudiante identificaba la causa

5. Decisiones donde más énfasis se puso

¿DAL o no DAL? Fue la decisión arquitectónica más debatida de esta etapa. Se analizaron pros y contras a lo largo de varios intercambios, concluyendo que para tres modelos con queries simples la separación en una capa DAL era sobreingeniería. El análisis se documentó con una tabla de criterios.

¿OAuth2PasswordBearer o HTTPBearer? Surgió cuando el Authorize de Swagger comenzó a fallar. Requerí entender la diferencia entre el flujo OAuth2 completo y el uso simple de Bearer tokens; la conclusión fue que OAuth2PasswordBearer implicaba un contrato que el sistema no cumplía.

¿security.py separado de deps.py? Generó debate sobre si valía la pena separar las funciones criptográficas de las dependencias de FastAPI. Se implementó porque la separación de responsabilidades tiene valor incluso en proyectos pequeños: si cambia el algoritmo de hashing, solo se toca security.py.

¿Teléfono como VARCHAR o INTEGER? Un ejemplo concreto y aparentemente trivial de por qué el tipo de dato importa: los teléfonos en Argentina empiezan con 0 o +54, y un INTEGER descartaría esos caracteres. VARCHAR es el tipo correcto porque no se realizan operaciones matemáticas con números de teléfono.

Parte II — Features Incrementales (v1.5.0 → v1.9.0)

6. Narrativa del proceso

El desarrollo retomó la base funcional en v1.5.0 y avanzó hasta v1.9.0 mediante cinco features incrementales, cada una en su propia rama de Git Flow, con Pull Request documentado e integración ordenada hacia develop y main.

La dinámica se mantuvo idéntica a la de la Parte I: antes de escribir código se discutía el diseño, y antes de tomar una decisión se analizaban las alternativas disponibles. Las features cubiertas fueron manejo global de errores, paginación, filtros de búsqueda, estadísticas públicas, promoción de usuarios a administrador y recuperación de contraseña.

7. Decisiones de arquitectura

Decisión	Alternativas evaluadas	Decisión tomada	Razón
Ubicación de error handlers	Router / Service / Core	app/core/error_handlers.py	Infraestructura transversal, no pertenece a ningún dominio
Formato de respuesta de error	{detail} / {success,error} / {error}	{error:{type,message,details}}	Eliminar redundancia; el status code ya indica el resultado
Campo success:false	Incluirlo / Omitirlo	Omitido	El status HTTP + error.type ya comunican el fallo
Handler para ValueError	Try/except en router / HTTPException en service / Handler global	Handler global	DRY, red de seguridad, service desacoplado de FastAPI
Schema de paginación	En animal.py / Archivo propio	app/schemas/pagination.py	Concepto transversal, reutilizable en cualquier entidad
Parámetros de paginación	limit/offset / page/size	page/size	Más intuitivo para el cliente; el servidor calcula el offset
unique=True en recuperacion_password	Con unique / Sin unique	Con unique	La DB refuerza la invariancia del dominio, no solo el service
Modelo de recuperación de password	Campo en Usuario / Tabla separada	Tabla separada	Ciclo de vida diferente: entidad temporal vs. permanente

8. Dónde la IA aportó más valor

Diseño antes de código

En cada feature, la IA formuló preguntas antes de permitir escribir código: ¿Dónde viviría este archivo?, ¿Quién debería armar el PaginatedResponse?, ¿Por qué ese orden?. Esto forzó a fundamentar cada decisión antes de implementarla.

Detección de inconsistencias

Cuando el import en main.py no coincidía con el nombre de la función en error_handlers.py, la IA lo detectó antes de que el servidor fallara. Cuando una rama se creó desde main en lugar de develop, la IA lo cuestionó antes de avanzar con el desarrollo.

Preguntas de diseño no obvias

La discusión sobre success:false como campo redundante, la diferencia entre limit/offset y page/size, y la decisión de usar unique=True en la tabla de recuperación de contraseña fueron momentos donde la IA planteó el problema y el razonamiento de la solución se construyó de forma autónoma.

9. Dónde se invirtió más tiempo

Conflictos de Alembic (múltiples heads)

El historial de migraciones presentó dos heads divergentes, generado por migraciones creadas en ramas paralelas sin verificar el estado previo. Se resolvió con alembic merge heads, estableciendo como hábito correr alembic heads antes de cada nueva migración.

Conflictos de merge entre develop y main

Los archivos de configuración de Alembic (alembic.ini, env.py, script.py.mako) generaron conflictos por tener versiones distintas en cada rama. Se resolvieron localmente en VS Code, aceptando la versión más completa en cada caso, y commiteando la resolución antes de abrir el Pull Request.

Debugging del flujo de recuperación de contraseña

El endpoint devolvía siempre el mensaje genérico de éxito aunque el flujo interno fallara en detectarse. La causa fue que logger.info no se mostraba en la consola de uvicorn sin configuración explícita de logging. Se diagnosticó agregando prints temporales paso a paso hasta aislar el punto exacto, y se resolvió estructurando un email_service.py como capa de abstracción, fácil de reemplazar cuando se integre Mailtrap.

10. Decisiones tomadas de forma independiente en esta etapa

- Detectar que una rama de feature se había creado desde main en lugar de develop
- Proponer el mapping HTTP_ERROR_TYPES para diferenciar tipos de error dentro de HTTPException
- Identificar que success:false era información redundante frente al status code HTTP
- Proponer app/schemas/pagination.py como archivo independiente y reutilizable
- Diseñar el modelo RecuperacionPassword con unique=True, justificándolo con argumentos de integridad de dominio
- Decidir que ValueError debía manejarse en un handler global en lugar de en cada router individual

- Proponer el `email_service` como capa de abstracción para facilitar la integración futura con Mailtrap

Cierre del Proyecto

Herramientas de IA utilizadas

- Claude (Anthropic) — mentor técnico durante todo el desarrollo del backend, desde la arquitectura inicial hasta la última feature incremental

Tipo de asistencia recibida

- Preguntas conceptuales y de diseño antes de cada implementación
- Validación de decisiones arquitectónicas ya razonadas de forma independiente
- Detección de inconsistencias entre archivos y configuraciones
- Guía en debugging mediante preguntas, sin proveer la solución directa
- Revisión de código antes de cada commit
- Simulación de pair programming, code review y entrevista técnica en distintas etapas

Qué se desarrolló de forma independiente

- Todo el código de la aplicación, en ambas etapas del proyecto
- Todas las decisiones de diseño y arquitectura
- El razonamiento y la justificación detrás de cada decisión
- La detección y corrección de los errores encontrados durante el desarrollo
- La configuración completa del entorno de testing y su posterior depuración
- El flujo completo de control de versiones: ramas, Pull Requests y tags semánticos

Reflexión final

Los momentos de más fricción a lo largo de todo el proyecto —especialmente en testing, Git Flow y la configuración de Alembic— fueron los más valiosos. No porque fueran agradables, sino porque los errores resueltos de forma independiente son los que menos se van a repetir.

Al cierre del proyecto, cada decisión arquitectónica de AdoptAR —desde la elección de una sola tabla Usuario hasta la tabla separada para la recuperación de contraseña— puede defenderse con un razonamiento concreto. No porque se haya enseñado, sino porque se razonó.